

CHAPTER 24

More Supervised Machine Learning Techniques with Scikit-learn

This chapter will cover using Scikit-learn to implement machine learning models using techniques such as

- Feature engineering
- Resampling methods
- Model evaluation methods
- Pipelines for streamlining machine learning workflows
- Techniques for model tuning

Feature Engineering

Feature engineering is the process of systematically choosing the set of features in the dataset that are useful and relevant to the learning problem. It is often the case that irrelevant features negatively affect the performance of the model. This section will review some techniques implemented in Scikit-learn for selecting relevant features from a dataset. The techniques surveyed include

- Statistical tests to select the best k features using the **SelectKBest** module
- Recursive feature elimination (RFE) to recursively remove irrelevant features from the dataset

- Principal component analysis to select the components that account for the variation in the dataset
- Feature importances using ensembled or tree classifiers

Statistical Tests to Select the Best k Features Using the `SelectKBest` Module

The following list is a selection of statistical tests to use with **SelectKBest**. The choice depends if the dataset target variable is numerical or categorical:

- ANOVA F-value, **f_classif** (classification)
- Chi-squared stats of non-negative features, **chi2** (classification)
- F-value, **f_regression** (regression)
- Mutual information for a continuous target, **mutual_info_regression** (regression)

Let's see an example using chi-squared test to select the best variables.

```
# import packages
from sklearn import datasets
from sklearn.feature_selection import SelectKBest
from sklearn.feature_selection import chi2

# load dataset
data = datasets.load_iris()

# separate features and target
X = data.data
y = data.target

# display first 5 rows
X[0:5,:]

# feature engineering. Let's see the best 3 features by setting k = 3
kBest_chi = SelectKBest(score_func=chi2, k=3)
fit_test = kBest_chi.fit(X, y)
```

```
# print test scores
fit_test.scores_
'Output': array([ 10.81782088,   3.59449902, 116.16984746,  67.24482759])
```

From the test scores, the top 3 important features in the dataset are ranked from feature 3 to 4 to 1 and to 2 in order. The data scientist can choose to drop the second column and observe the effect on the model performance.

We can transform the dataset to subset only the important features.

```
adjusted_features = fit_test.transform(X)
adjusted_features[0:5,:]
'Output':
array([[5.1, 1.4, 0.2],
       [4.9, 1.4, 0.2],
       [4.7, 1.3, 0.2],
       [4.6, 1.5, 0.2],
       [5. , 1.4, 0.2]])
```

The result drops the second column of the dataset.

Recursive Feature Elimination (RFE)

RFE is used together with a learning model to recursively select the desired number of top performing features.

Let's use RFE with **LinearRegression**.

```
# import packages
from sklearn.feature_selection import RFE
from sklearn.linear_model import LinearRegression
from sklearn import datasets

# load dataset
data = datasets.load_boston()

# separate features and target
X = data.data
y = data.target
```

```
# feature engineering
linear_reg = LinearRegression()
rfe = RFE(estimator=linear_reg, n_features_to_select=6)
rfe_fit = rfe.fit(X, y)

# print the feature ranking
rfe_fit.ranking_
'Output': array([3, 5, 4, 1, 1, 1, 8, 1, 2, 6, 1, 7, 1])
```

From the result, the 4th, 5th, 6th, 8th, 11th, and 13th features are the top 6 features in the Boston dataset.

Feature Importances

Tree-based or ensemble methods in Scikit-learn have a **feature_importances_** attribute which can be used to drop irrelevant features in the dataset using the **SelectFromModel** module contained in the **sklearn.feature_selection** package.

Let's use the ensemble method **AdaBoostClassifier** in this example.

```
# import packages
from sklearn.ensemble import AdaBoostClassifier
from sklearn.feature_selection import SelectFromModel
from sklearn import datasets

# load dataset
data = datasets.load_iris()

# separate features and target
X = data.data
y = data.target

# original data shape
X.shape

# feature engineering
ada_boost_classifier = AdaBoostClassifier()
ada_boost_classifier.fit(X, y)
```

```

'Output':
AdaBoostClassifier(algorithm='SAMME.R', base_estimator=None,
                    learning_rate=1.0, n_estimators=50, random_state=None)

# print the feature importances
ada_boost_classifier.feature_importances_
'Output': array([0.   , 0.   , 0.58, 0.42])

# create a subset of data based on the relevant features
model = SelectFromModel(ada_boost_classifier, prefit=True)
new_data = model.transform(X)

# the irrelevant features have been removed
new_data.shape
'Output': (150, 2)

```

Resampling Methods

Resampling methods are a set of techniques that involve selecting a subset of the available dataset, training on that data subset, and using the remainder of the data to evaluate the trained model. Let's review the techniques for resampling using Scikit-learn. This section covers

- k-Fold cross-validation
- Leave-one-out cross-validation

k-Fold Cross-Validation

In k-fold cross validation, the dataset is divided into k-parts or folds. The model is trained using $k - 1$ folds and evaluated on the remaining kth fold. This process is repeated k-times so that each fold can serve as a test set. At the end of the process, k-fold averages the result and reports a mean score with a standard deviation. Scikit-learn implements K-fold CV in the module **KFold**. The module **cross_val_score** is used to evaluate the cross-validation score using the **splitting** strategy, which is **KFold** in this case.

Let's see an example of this using the k-nearest neighbors (kNN) classification algorithm. When initializing **KFold**, it is standard practice to shuffle the data before splitting.

```
from sklearn.model_selection import KFold
from sklearn.model_selection import cross_val_score
from sklearn.neighbors import KNeighborsClassifier

# load dataset
data = datasets.load_iris()

# separate features and target
X = data.data
y = data.target

# initialize KFold - with shuffle = True, shuffle the data before splitting
kfold = KFold(n_splits=3, shuffle=True)

# create the model
knn_clf = KNeighborsClassifier(n_neighbors=3)

# fit the model using cross validation
cv_result = cross_val_score(knn_clf, X, y, cv=kfold)

# evaluate the model performance using accuracy metric
print("Accuracy: %.3f%% (%.3f%%)" % (cv_result.mean()*100.0, cv_result.
std()*100.0))
'Output':
Accuracy: 93.333% (2.494%)
```

Leave-One-Out Cross-Validation (LOOCV)

In LOOCV just one example is assigned to the test set, and the model is trained on the remainder of the dataset. This process is repeated for all the examples in the dataset. This process is repeated until all the examples in the dataset have been used for evaluating the model.

```
from sklearn.model_selection import LeaveOneOut
from sklearn.model_selection import cross_val_score
from sklearn.neighbors import KNeighborsClassifier
```

```

# load dataset
data = datasets.load_iris()

# separate features and target
X = data.data
y = data.target

# initialize LOOCV
loocv = LeaveOneOut()

# create the model
knn_clf = KNeighborsClassifier(n_neighbors=3)

# fit the model using cross validation
cv_result = cross_val_score(knn_clf, X, y, cv=loocv)

# evaluate the model performance using accuracy metric
print("Accuracy: %.3f%% (%.3f%%)" % (cv_result.mean()*100.0, cv_result.
std()*100.0))
'Output':
Accuracy: 96.000% (19.596%)

```

Model Evaluation

This chapter has already used a couple of evaluation metrics for assessing the quality of the fitted models. In this section, we survey a couple of other metrics for regression and classification use cases and how to implement them using Scikit-learn. For each metric, we show how to use them as stand-alone implementations, as well as together with cross-validation using the **cross_val_score** method.

What we'll cover here includes

Regression evaluation metrics

- Mean squared error (MSE): The average sum of squared difference between the predicted label, $\hat{y}$, and the true label, y . A score of 0 indicates a perfect prediction without errors.
- Mean absolute error (MAE): The average absolute difference between the predicted label, $\hat{y}$, and the true label, y . A score of 0 indicates a perfect prediction without errors.

- R^2 : The amount of variance or variability in the dataset explained by the model. The score of 1 means that the model perfectly captures the variability in the dataset.

Classification evaluation metrics

- Accuracy: Is the ratio of correct predictions to the total number of predictions. The bigger the accuracy, the better the model.
- Logarithmic loss (a.k.a logistic loss or cross-entropy loss): Is the probability that an observation is correctly assigned to a class label. By minimizing the log-loss, conversely, the accuracy is maximized. So with this metric, values closer to zero are good.
- Area under the ROC curve (AUC-ROC): Used in the binary classification case. Implementation is not provided, but very similar in style to the others.
- Confusion matrix: More intuitive in the binary classification case. Implementation is not provided, but very similar in style to the others.
- Classification report: It returns a text report of the main classification metrics.

Regression Evaluation Metrics

The following code is an example of regression evaluation metrics implemented stand-alone.

```
# import packages
from sklearn.linear_model import LinearRegression
from sklearn import datasets
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error
from sklearn.metrics import mean_absolute_error
from sklearn.metrics import r2_score

# load dataset
data = datasets.load_boston()
```



```

# separate features and target
X = data.data
y = data.target

# split in train and test sets
X_train, X_test, y_train, y_test = train_test_split(X, y, shuffle=True)

# create the model
# setting normalize to true normalizes the dataset before fitting the model
linear_reg = LinearRegression(normalize = True)

# fit the model on the training set
linear_reg.fit(X_train, y_train)
'Output': LinearRegression(copy_X=True, fit_intercept=True, n_jobs=1,
normalize=True)

# make predictions on the test set
predictions = linear_reg.predict(X_test)
# evaluate the model performance using mean square error metric
print("Mean squared error: %.2f" % mean_squared_error(y_test, predictions))
'Output':
Mean squared error: 14.46

# evaluate the model performance using mean absolute error metric
print("Mean absolute error: %.2f" % mean_absolute_error(y_test,
predictions))
'Output':
Mean absolute error: 3.63

# evaluate the model performance using r-squared error metric
print("R-squared score: %.2f" % r2_score(y_test, predictions))
'Output':
R-squared score: 0.69

```

The following code is an example of regression evaluation metrics implemented with cross-validation. The MSE and MAE metrics for cross-validation are implemented with the sign inverted. The simple way to interpret this is to have it in mind that the closer the values are to zero, the better the model.

```

from sklearn.linear_model import LinearRegression
from sklearn.model_selection import KFold
from sklearn.model_selection import cross_val_score

# load dataset
data = datasets.load_boston()

# separate features and target
X = data.data
y = data.target

# initialize KFold - with shuffle = True, shuffle the data before splitting
kfold = KFold(n_splits=3, shuffle=True)

# create the model
linear_reg = LinearRegression(normalize = True)

# fit the model using cross validation - score with Mean square error (MSE)
mse_cv_result = cross_val_score(linear_reg, X, y, cv=kfold, scoring="neg_
mean_squared_error")
# print mse cross validation output
print("Negative Mean squared error: %.3f%% (%.3f%%)" % (mse_cv_result.
mean(), mse_cv_result.std()))
'Output':
Negative Mean squared error: -24.275% (4.093%)

# fit the model using cross validation - score with Mean absolute error (MAE)
mae_cv_result = cross_val_score(linear_reg, X, y, cv=kfold, scoring="neg_
mean_absolute_error")
# print mse cross validation output
print("Negative Mean absolute error: %.3f%% (%.3f%%)" % (mae_cv_result.
mean(), mae_cv_result.std()))
'Output':
Negative Mean absolute error: -3.442% (4.093%)

# fit the model using cross validation - score with R-squared
r2_cv_result = cross_val_score(linear_reg, X, y, cv=kfold, scoring="r2")
# print mse cross validation output

```

```
print("R-squared score: %.3f%% (%.3f%%)" % (r2_cv_result.mean(), r2_cv_
result.std()))
'Output':
R-squared score: 0.707% (0.030%)
```

Classification Evaluation Metrics

The following code is an example of classification evaluation metrics implemented stand-alone.

```
# import packages
from sklearn.linear_model import LogisticRegression
from sklearn import datasets
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score
from sklearn.metrics import log_loss
from sklearn.metrics import classification_report

# load dataset
data = datasets.load_iris()

# separate features and target
X = data.data
y = data.target

# split in train and test sets
X_train, X_test, y_train, y_test = train_test_split(X, y, shuffle=True)

# create the model
logistic_reg = LogisticRegression()

# fit the model on the training set
logistic_reg.fit(X_train, y_train)
'Output':
LogisticRegression(C=1.0, class_weight=None, dual=False, fit_intercept=True,
                    intercept_scaling=1, max_iter=100, multi_class='ovr', n_jobs=1,
                    penalty='l2', random_state=None, solver='liblinear', tol=0.0001,
                    verbose=0, warm_start=False)
```

```

# make predictions on the test set
predictions = logistic_reg.predict(X_test)

# evaluate the model performance using accuracy
print("Accuracy score: %.2f" % accuracy_score(y_test, predictions))
'Output':
Accuracy score: 0.89

# evaluate the model performance using log loss

### output the probabilities of assigning an observation to a class
predictions_probabilities = logistic_reg.predict_proba(X_test)
print("Log-Loss likelihood: %.2f" % log_loss(y_test, predictions_
probabilities))
'Output':
Log-Loss likelihood: 0.39

# evaluate the model performance using classification report
print("Classification report: \n", classification_report(y_test,
predictions, target_names=data.target_names))
'Output':
Classification report:

```

	precision	recall	f1-score	support
setosa	1.00	1.00	1.00	12
versicolor	0.85	0.85	0.85	13
virginica	0.85	0.85	0.85	13
avg / total	0.89	0.89	0.89	38

Let's see an example of classification evaluation metrics implemented with cross-validation. Evaluation metrics for log-loss using cross-validation is implemented with the sign inverted. The simple way to interpret this is to have it in mind that the closer the values are to zero, the better the model.

```

from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import KFold
from sklearn.model_selection import cross_val_score

```

```

# load dataset
data = datasets.load_iris()

# separate features and target
X = data.data
y = data.target

# initialize KFold - with shuffle = True, shuffle the data before splitting
kfold = KFold(n_splits=3, shuffle=True)

# create the model
logistic_reg = LogisticRegression()

# fit the model using cross validation - score with accuracy
accuracy_cv_result = cross_val_score(logistic_reg, X, y, cv=kfold,
scoring="accuracy")
# print accuracy cross validation output
print("Accuracy: %.3f%% (%.3f%%)" % (accuracy_cv_result.mean(), accuracy_
cv_result.std()))
'Output':
Accuracy: 0.953% (0.025%)

# fit the model using cross validation - score with Log-Loss
logloss_cv_result = cross_val_score(logistic_reg, X, y, cv=kfold,
scoring="neg_log_loss")
# print mse cross validation output
print("Log-Loss likelihood: %.3f%% (%.3f%%)" % (logloss_cv_result.mean(),
logloss_cv_result.std()))
'Output':
Log-Loss likelihood: -0.348% (0.027%)

```

Pipelines: Streamlining Machine Learning Workflows

The concept of pipelines in Scikit-learn is a compelling tool for chaining a bunch of operations together to form a tidy process flow of data transforms from one state to another. The operations that constitute a pipeline can be any of Scikit-learn's

transformers (i.e., modules with a **fit** and **transform** method, or a **fit_transform** method) or classifiers (i.e., modules with a **fit** and **predict** method, or a **fit_predict** method). Classifiers are also called predictors.

For a typical machine learning workflow, the steps taken may involve cleaning the data, feature engineering, scaling the dataset, and then fitting a model. Pipelines can be used in this case to chain these operations together into a coherent workflow. They have the advantage of providing a convenient and consistent interface for calling at once a sequence of operations.

These transformers or predictors are collectively called estimators in Scikit-learn terminology. In the last two paragraphs, we called them operations.

Another advantage of pipelines is that it safeguards against accidentally fitting a transform on the entire dataset and thereby leaking statistics influenced by the test data to the machine learning model while training. For example, if a standardizer is fitted on the whole dataset, the test set will be compromised because the test observations have contributed in estimating the mean and standard deviation for scaling the training set before fitting the model.

Finally, only the last step of the pipeline can be a classifier or predictor. All the stages of the pipeline must contain a **transform** method except the final stage, which can be a transformer or a classifier.

To begin using Scikit-learn pipelines, first import

```
from sklearn.pipeline import Pipeline
```

Let's see some examples of working with Pipelines in Scikit-learn. In the following example, we'll apply a scaling transform to standardize our dataset and then use a support vector classifier to train the model.

```
# import packages
from sklearn.svm import SVC
from sklearn import datasets
from sklearn.model_selection import KFold
from sklearn.model_selection import cross_val_score
from sklearn.preprocessing import StandardScaler

from sklearn.pipeline import Pipeline

# load dataset
data = datasets.load_iris()
```

```

# separate features and target
X = data.data
y = data.target

# create the pipeline
estimators = [
    ('standardize' , StandardScaler()),
    ('svc', SVC())
]

# build the pipeline model
pipe = Pipeline(estimators)

# run the pipeline
kfold = KFold(n_splits=3, shuffle=True)
cv_result = cross_val_score(pipe, X, y, cv=kfold)

# evaluate the model performance
print("Accuracy: %.3f%% (%.3f%%)" % (cv_result.mean()*100.0, cv_result.
std()*100.0))
'Output':
Accuracy: 94.667% (0.943%)

```

Pipelines Using `make_pipeline`

Another method for building machine learning pipelines is by using the **`make_pipeline`** method. For the next example, we use PCA to select the best six features and reduce the dimensionality of the dataset, and then we'll fit the model using Random forests for regression.

```

from sklearn.pipeline import make_pipeline
from sklearn.svm import SVR
from sklearn import datasets
from sklearn.model_selection import KFold
from sklearn.model_selection import cross_val_score
from sklearn.decomposition import PCA
from sklearn.pipeline import Pipeline
from sklearn.ensemble import RandomForestRegressor

```

```

# load dataset
data = datasets.load_boston()

# separate features and target
X = data.data
y = data.target

# build the pipeline model
pipe = make_pipeline(
    PCA(n_components=9),
    RandomForestRegressor()
)

# run the pipeline
kfold = KFold(n_splits=4, shuffle=True)
cv_result = cross_val_score(pipe, X, y, cv=kfold)

# evaluate the model performance
print("Accuracy: %.3f%% (%.3f%%)" % (cv_result.mean()*100.0, cv_result.
std()*100.0))
'Output':
Accuracy: 73.750% (2.489%)

```

Pipelines Using FeatureUnion

Scikit-learn provides a module for merging the output of several transformers called **feature_union**. It does this by fitting each transformer independently to the dataset, and then their respective outputs are combined to form a transformed dataset for training the model.

FeatureUnion works in the same way as a Pipeline, and in many ways can be thought of as a means of building complex pipelines within a Pipeline.

Let's see an example using FeatureUnion. Here, we will combine the output of recursive feature elimination (RFE) and PCA for feature engineering, and then we'll apply the Stochastic Gradient Boosting (SGB) ensemble model for regression to train the model.

```

from sklearn.ensemble import GradientBoostingRegressor
from sklearn.ensemble import RandomForestRegressor
from sklearn import datasets

```



```

from sklearn.model_selection import KFold
from sklearn.model_selection import cross_val_score
from sklearn.feature_selection import RFE
from sklearn.decomposition import PCA
from sklearn.pipeline import Pipeline
from sklearn.pipeline import make_pipeline
from sklearn.pipeline import make_union

# load dataset
data = datasets.load_boston()

# separate features and target
X = data.data
y = data.target

# construct pipeline for feature engineering - make_union similar to make_
pipeline
feature_engr = make_union(
    RFE(estimator=RandomForestRegressor(n_estimators=100), n_features_to_
        select=6),
    PCA(n_components=9)
)

# build the pipeline model
pipe = make_pipeline(
    feature_engr,
    GradientBoostingRegressor(n_estimators=100)
)

# run the pipeline
kfold = KFold(n_splits=4, shuffle=True)
cv_result = cross_val_score(pipe, X, y, cv=kfold)

# evaluate the model performance
print("Accuracy: %.3f%% (%.3f%%)" % (cv_result.mean()*100.0, cv_result.
std()*100.0))
'Output':
Accuracy: 88.956% (1.493%)

```

Model Tuning

Each machine learning model has a set of options or configurations that can be tuned to optimize the model when fitting to data. These configurations are called **hyper-parameters**. Hence, for each hyper-parameter, there exist a range of values that can be chosen. Taking into consideration the number of hyper-parameters that an algorithm has, the entire space can become exponentially large and infeasible to explore all of them. Scikit-learn provides two convenient modules for searching through the hyper-parameter space of an algorithm to find the best values for each hyper-parameter that optimizes the model.

These modules are the

- Grid search
- Randomized search

Grid Search

Grid search comprehensively explores all the specified hyper-parameter values for an estimator. It is implemented using the **GridSearchCV** module. Let's see an example using the Random forest for regression. The hyper-parameters we'll search over are

- The number of trees in the forest, **n_estimators**
- The maximum depth of the tree, **max_depth**
- The minimum number of samples required to split an internal node, **min_samples_leaf**

```
from sklearn.model_selection import GridSearchCV
from sklearn.ensemble import RandomForestRegressor
from sklearn import datasets

# load dataset
data = datasets.load_boston()

# separate features and target
X = data.data
y = data.target
```

```

# construct grid search parameters in a dictionary
parameters = {
    'n_estimators': [2, 4, 6, 8, 10, 12, 14, 16],
    'max_depth': [2, 4, 6, 8],
    'min_samples_leaf': [1,2,3,4,5]
}

# create the model
rf_model = RandomForestRegressor()

# run the grid search
grid_search = GridSearchCV(estimator=rf_model, param_grid=parameters)

# fit the model
grid_search.fit(X,y)
'Output':
GridSearchCV(cv=None, error_score='raise',
             estimator=RandomForestRegressor(bootstrap=True, criterion='mse',
             max_depth=None,
             max_features='auto', max_leaf_nodes=None,
             min_impurity_decrease=0.0, min_impurity_split=None,
             min_samples_leaf=1, min_samples_split=2,
             min_weight_fraction_leaf=0.0, n_estimators=10, n_jobs=1,
             oob_score=False, random_state=None, verbose=0, warm_
             start=False),
             fit_params=None, iid=True, n_jobs=1,
             param_grid={'n_estimators': [2, 4, 6, 8, 10, 12, 14, 16],
             'max_depth': [2, 4, 6, 8], 'min_samples_leaf': [1, 2, 3, 4, 5]},
             pre_dispatch='2*n_jobs', refit=True, return_train_score='warn',
             scoring=None, verbose=0)

# evaluate the model performance
print("Best Accuracy: %.3f%%" % (grid_search.best_score_*100.0))
'Output':
Best Accuracy: 57.917%

```

```
# best set of hyper-parameter values
print("Best n_estimators: %d \nBest max_depth: %d \nBest min_samples_leaf:
%d " % \
      (grid_search.best_estimator_.n_estimators, \
       grid_search.best_estimator_.max_depth, \
       grid_search.best_estimator_.min_samples_leaf))

'Output':
Best n_estimators: 14
Best max_depth: 8
Best min_samples_leaf: 1
```

Randomized Search

As opposed to grid search, not all the provided hyper-parameter values are evaluated, but rather a determined number of hyper-parameter values are sampled from a random uniform distribution. The number of hyper-parameter values that can be evaluated is determined by the **n_iter** attribute of the **RandomizedSearchCV** module.

In this example, we will use the same scenario as in the grid search case.

```
from sklearn.model_selection import RandomizedSearchCV
from sklearn.ensemble import RandomForestRegressor
from sklearn import datasets

# load dataset
data = datasets.load_boston()

# separate features and target
X = data.data
y = data.target

# construct grid search parameters in a dictionary
parameters = {
    'n_estimators': [2, 4, 6, 8, 10, 12, 14, 16],
    'max_depth': [2, 4, 6, 8],
    'min_samples_leaf': [1,2,3,4,5]
}
```

```

# create the model
rf_model = RandomForestRegressor()

# run the grid search
randomized_search = RandomizedSearchCV(estimator=rf_model, param_
distributions=parameters, n_iter=10)
# fit the model
randomized_search.fit(X,y)
'Output':
RandomizedSearchCV(cv=None, error_score='raise',
    estimator=RandomForestRegressor(bootstrap=True, criterion='mse',
    max_depth=None,
    max_features='auto', max_leaf_nodes=None,
    min_impurity_decrease=0.0, min_impurity_split=None,
    min_samples_leaf=1, min_samples_split=2,
    min_weight_fraction_leaf=0.0, n_estimators=10, n_jobs=1,
    oob_score=False, random_state=None, verbose=0, warm_
    start=False),
    fit_params=None, iid=True, n_iter=10, n_jobs=1,
    param_distributions={'n_estimators': [2, 4, 6, 8, 10, 12, 14, 16],
    'max_depth': [2, 4, 6, 8], 'min_samples_leaf': [1, 2, 3, 4, 5]},
    pre_dispatch='2*n_jobs', random_state=None, refit=True,
    return_train_score='warn', scoring=None, verbose=0)

# evaluate the model performance
print("Best Accuracy: %.3f%%" % (randomized_search.best_score_*100.0))
'Output':
Best Accuracy: 57.856%

# best set of hyper-parameter values
print("Best n_estimators: %d \nBest max_depth: %d \nBest min_samples_leaf:
%d " % \
    (randomized_search.best_estimator_.n_estimators, \
    randomized_search.best_estimator_.max_depth, \
    randomized_search.best_estimator_.min_samples_leaf))

```

```
'Output':  
Best n_estimators: 12  
Best max_depth: 6  
Best min_samples_leaf: 5
```

This chapter further explored using Scikit-learn to incorporate other machine learning techniques such as feature selection and resampling methods to develop a more robust machine learning method. In the next chapter, we will examine our first unsupervised machine learning method, clustering, and its implementation with Scikit-learn.